Chaîne alimentaire

Exercice de synthèse

Consignes générales :

- ⇒ **L'IDE à utiliser est PyCharm.**
- ⇒ Il est fortement recommandé d'ouvrir les supports de cours Python pour vérifier les définitions et les syntaxes.
- ⇒ Toutes les fonctions sont à écrire dans un fichier séparés appelé **functions.py**
- ⇒ Tous les tests des fonctions seront à faire dans le fichier **main.py**
- ⇒ Les instructions **print()** et **input()** sont interdites dans les fonctions sauf si le rôle même de la fonction est de permettre l'affichage ou la saisie d'informations utilisateur.

Acquis d'apprentissage :

A la fin de ce TP, chaque étudiant doit être capable de s'autoévaluer et d'identifier les notions qu'il maîtrise en Python et celles qu'il doit travailler davantage.

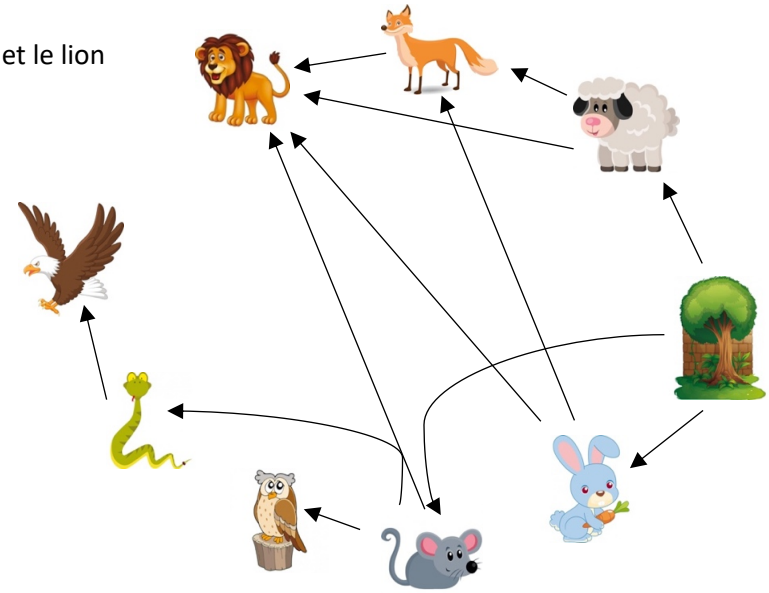
Présentation du problème

L'objectif de ce problème est d'implémenter les interactions d'une chaîne alimentaire. Il s'agit de travailler deux aspects :

- Manipulation des fichiers
- Manipulation des structures de données.

L'image[1] ci-dessous représente la chaîne alimentaire de plusieurs animaux. La flèche → est dirigée de la proie vers le prédateur. Ainsi, l'on peut lire :

- Le mouton consomme de l'herbe
- Le mouton peut être mangé par le chacal et le lion



[1] Les images ont été tirées de diverses sources du web pour illustrer uniquement le problème décrit dans ce TP.

Pour représenter cette interaction, nous allons utiliser une matrice carrée (nombre de lignes = nombre de colonnes). Les lignes et les colonnes vont représenter les éléments de notre image, ce qui crée dans notre cas une matrice carré de 9 x 9. Chaque case (i, j) de la matrice est remplie comme suit :

- -1 Si i = proie et j = prédateur de la proie i
- 1 Si i = prédateur et j = proie du prédateur i
- 0 Si pas de relation proie / prédateur entre i et j

La représentation matricielle de l'image ci-dessus est donnée dans le fichier **chaine.txt** fourni sous Moodle. Le contenu de ce fichier se lit comme suit :

- 1ère ligne : représente la dimension de la matrice (nombre de lignes = nombre de colonnes)
- 2ème ligne : représente les éléments de la chaîne alimentaire listés dans l'ordre.
- À partir de la 3ème ligne : lignes de la matrice où les valeurs sont séparées par des espaces. Les colonnes correspondent aux éléments décrits à la 2ème ligne du même fichier.

Représentation de la chaîne alimentaire

Exercice 1

Écrire une fonction qui permet de :

- Lire le fichier **chaine.txt** et extraire les noms des éléments de la chaîne alimentaire.
- Retourner deux dictionnaires :
  - D1 : dans lequel chacun des éléments de la chaîne alimentaire sera considéré comme une clé associée à une valeur (de type entier) séquentielle allant de 0 à 8. Les éléments sont à considérer dans l'ordre qui a été donné dans le fichier.

    {'lion': 0, 'chacal': 1, 'mouton': 2, 'arbre': 3, 'lapin': 4, 'souris': 5, 'hibou': 6, 'serpent': 7, 'aigle': 8}

  - D2 : dans lequel chacun des éléments de la chaîne alimentaire sera considéré comme une valeur associée à une clé (de type entier) séquentielle allant de 0 à 8. Les éléments sont à considérer dans l'ordre qui a été donné dans le fichier.

    {0: 'lion', 1: 'chacal', 2: 'mouton', 3: 'arbre', 4: 'lapin', 5: 'souris', 6: 'hibou', 7: 'serpent', 8: 'aigle'}

Exercice 2

Écrire une fonction qui prend en paramètres une liste à 2D d'entiers et qui permet de la remplir à partir de la représentation matricielle de l'image ci-dessus se trouvant dans le fichier **chaine.txt**.

	0	1	1	0	1	1	0	0	0
	-1	0	1	0	1	0	0	0	0
	-1	-1	0	1	0	0	0	0	0
	0	0	-1	0	-1	-1	0	0	0
	-1	-1	0	1	0	0	0	0	0
	-1	0	0	1	0	0	-1	-1	0
	0	0	0	0	0	1	0	0	0
	0	0	0	0	0	1	0	0	-1
	0	0	0	0	0	0	0	1	0

Exercice 3

Écrire une fonction qui prend en paramètres un dictionnaire (l'un des dictionnaires retournés à l'exercice 1) et une liste à 2D d'entiers (remplie à l'exercice 2) et qui permet de retourner un dictionnaire prédateur où les clés représentent les éléments de l'images et les valeurs seront les listes de leurs proies.

```
chaine_ali_predateur = {"lion": ["chacal", "mouton", "lapin", "souris"],
        "chacal": ["mouton", "lapin"],
        ....
        }
```

Exercice 4

Écrire une fonction qui prend en paramètres un dictionnaire (l'un des dictionnaires retournés à l'exercice 1) et une liste à 2D d'entiers (remplie à l'exercice 2) et qui permet de retourner un dictionnaire proie où les clés représentent les éléments de l'images et les valeurs seront les listes de leurs prédateurs.

```
chaine_ali_proie = {"lion": [],
        "chacal": ["lion"],
        "mouton":["chacal", "lion"],
         ...
        }
```

Opérations sur la chaîne alimentaire

Exercice 1

Écrire une fonction qui prend en paramètres un dictionnaire prédateur et qui permet de mettre à jour le fichier **chaine.txt** en ajoutant les éléments du dictionnaire à la chaîne alimentaire.

Exemple :

```
predateur = {"chat_sauvage": ["lapin", "souris"]}
```

Exercice 2

Écrire une fonction qui prend en paramètres l'un des éléments de la chaîne alimentaire et qui permet de le supprimer et de mettre à jour le fichier **chaine.txt**.

Exercice 3

Écrire une fonction qui prend en paramètres **la matrice 2D**, un dictionnaire (l'un des dictionnaires générés à l'exercice 1) , un élément de la chaîne alimentaire **elem** et un caractère de séparation **Sep** dont la valeur par défaut est **«** → ». Elle doit permettre de reconstituer la chaîne alimentaire de **elem**.

Indication : Utilisez la stratégie de votre choix.

Affichage attendu :

Si la fonction est appelée avec seulement 3 arguments : **Lion --> Chacal --> Lapin --> Arbre**

Si la fonction est appelée avec 4 arguments et Sep = « : » : **Lion : Chacal : Lapin : Arbre**

Exercice 4

En vous basant sur la fonction précédente, écrire une fonction qui prend en paramètres la matrice 2D, un dictionnaire (l'un des dictionnaires générés à l'exercice 1) et un caractère de séparation **Sep** dont la valeur par défaut est **«** → », et qui permet de reconstituer la chaîne alimentaire de chacun des éléments de la chaîne.

Indication : Utilisez la stratégie de votre choix.

Exemple d'affichage attendu :

Lion --> Chacal --> Lapin --> Arbre

Chacal --> Mouton --> Arbre

….

Exercice 5

Écrire une fonction qui prend en paramètres **un dictionnaire prédateur**, un dictionnaire (l'un des dictionnaires générés à l'exercice 1), un élément de la chaîne alimentaire **elem** et un caractère de séparation **Sep** dont la valeur par défaut est **«** → ». Elle doit permettre de reconstituer la chaîne alimentaire de **elem**.

Indication : Utilisez la stratégie de votre choix.

Affichage attendu :

Si la fonction est appelée avec seulement 3 arguments : **Lion --> Chacal --> Lapin --> Arbre**

Si la fonction est appelée avec 4 arguments et Sep = « : » : **Lion : Chacal : Lapin : Arbre**

Exercice 6

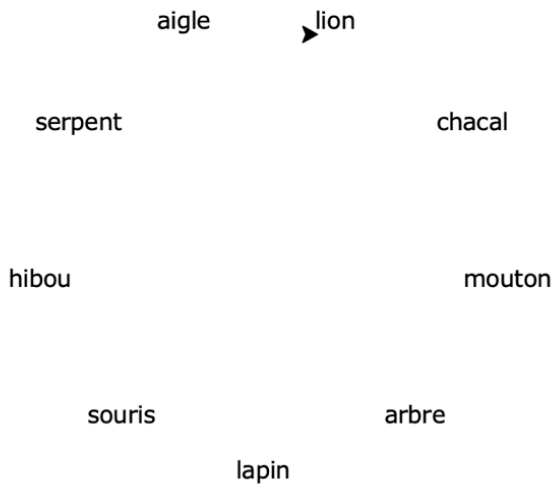
Dans un programme principal, tester les fonctions précédentes sous différents scénarios et vérifiez vos résultats.

Pour les plus motivés …

En utilisant le module Turtle, proposer une représentation graphique de la chaîne alimentaire.

Penser à paramétrer, au mieux, votre fonction.

Exemple de représentation (à compléter) obtenue avec la fonction write() de turtle :



Je m'auto-évalue :

⇒ Je sais manipuler des variables

⇒ Je maîtrise les boucles en Python (syntaxe + utilisation)

⇒ J'ai compris le fonctionnement des listes

⇒ J'ai compris le rôle des fonctions

⇒ Je peux écrire une fonction en Python en me basant sur la description de son prototype.

⇒ Je maîtrise l'appel des fonctions quel que soit leurs prototypes

⇒ Je peux faire la différence entre les types de collections

⇒ Je sais manipuler des listes 2D (matrices)

⇒ Je manipule facilement un dictionnaire

⇒ Je maîtrise l'utilisation des fichiers en Python.